Greenhouse Anomaly Detection in Environmental Sensor Data Using TinyML

Progress Report

Zhengliang “David” Song

1 Introduction

1.1 Background

Rewrite Text:

Modern greenhouses are increasingly reliant on the continuous observation and regulation of environmental parameters to maintain a stable and conducive growth environment. In reality, sensor data is rarely clean, and there are various ways in which faulty data could emerge, including but not limited to dissections, calibration, condensation, aging effects, noise artifacts, positioning errors, and physical environmental phenomena. When this occurs, decisions based on this data may be unreliable and could ultimately translate to a less-than-ideal growth environment.

Popular methods involve tracking sensor data from a central computer or cloud service to perform analysis. In the greenhouse setting, intermittent connectivity, reduced bandwidth, and a desire to be self-sufficient locally suggest edge or device-level anomaly detection, where anomalies are detected close to the source of the data. This matches TinyML, performing inference computations directly in resource-restricted devices to maintain responsiveness and robustness over challenged networks [3], [7].

Microclimates are spatially and temporally varied in greenhouses. Even for controlled ventilation systems, there may be temperature, humidity, or CO_2 concentration differences across the greenhouse structural boundaries [1]. This means the term “abnormal” must also be viewed in the proper context because what may seem abnormal may actually correspond to a legitimate local variation based on the airflows or control system operations rather than sensor malfunction. A sensoromaly detector must further differentiate sensor-related anomalies such as drift, stuck-at values, dropouts, or outlier values.

Emerging research suggests the capability for multivariate sensor streams to enable embedded intelligence for greenhouse monitoring and control. Ihoume et al. outline the TinyML modeling approach for greenhouse microclimate control based on multivariate sensor readings [2], illustrating the viability of embedded modeling for CO_2, temperature, and humidity. Control-oriented applications often rely on the premise that the input stream is reliable. The focus of the current project is on providing this complementary functionality: embedded anomaly detection for signaling sensor dysfunction to enable the differentiation of

sensor malfunction versus real-world occurrences for the purpose of trust in monitoring and control. Since the proposal stage, the technical direction has been honed to promote both feasibility and the rate of iteration and meet the goal of a low-power deployment [19]. The end hardware goal is the ESP32. Based on the advice received from the supervisor, the design will employ MicroPython to promote development implementation through the simulated development method of Wokwi and PlatformIO [10]. The method of deployment has changed from the original TensorFlow-centric concept to emlearn to suit the favorable implementation method for the microcontroller and the lightweight inference requirement [7].

1.2 Aim and objectives

1.2.1 Aim

The objective is to implement and test an embedded anomaly detection system for greenhouse sensor (CO2 levels, temperature, and relative humidity) readings. The embedded system must run on the ESP32 microcontroller and utilize the MicroPython programming language. The model must employ the emlearn deployable model. The embedded system's functionality and performance must be tested using simulators and hardware.

1.2.2 Objectives

1. Define requirements and success criteria for embedded anomaly detection in a greenhouse context, including detection performance, acceptable false-alarm behaviour, latency constraints, memory footprint, and power-related considerations.
2. Establish a data pipeline for CO_2, temperature, and humidity streams, including data logging/collection, cleaning, time alignment, and preprocessing suitable for model development and evaluation.
3. Select and justify an anomaly detection approach appropriate for constrained devices (e.g., lightweight unsupervised/threshold-based methods), including feature design and decision thresholds robust under realistic noise and drift conditions [3], [4].
4. Develop an end-to-end simulated prototype using Wokwi + PlatformIO to validate data flow, inference logic, and anomaly signalling behaviour using repeatable test scenarios [10].
5. Implement embedded inference on ESP32 in MicroPython, integrating an emlearn-based model representation and providing clear anomaly outputs (serial logs and a simple alert mechanism) [7].
6. Design and execute a test plan covering functional tests (normal vs abnormal scenarios), robustness checks, and embedded profiling (latency and memory use; power-related observations where measurable).

7. Evaluate and critically reflect on results, discussing limitations, constraints, and deployment considerations, and mapping achievements back to the objectives [20].

1.3 Product overview

The project artefact is a tiny anomaly detector for greenhouse environment monitoring that runs on-device. The tool monitors CO_2, temperature, and RH sensors, then outputs an anomaly determination for maintenance and monitoring. The project runs on an ESP32 development board with MicroPython because of its balance of internal functionality, ease of development, and ease of testing.

To support rapid iteration while maintaining a credible embedded target, development is staged:

- Simulation and rapid prototyping: Wokwi and PlatformIO validate embedded logic in a repeatable environment (deterministic scenarios and consistent test inputs) [10].
- Embedded deployment: the anomaly model is represented/deployed using emlearn and integrated into a MicroPython inference loop on ESP32 [7].
- Evaluation: the artefact is assessed using both simulated and hardware-based tests, with results reported against defined success criteria.

Planned deliverables include: a working ESP32 MicroPython implementation; an emlearn-based deployable model artefact and conversion/export pipeline; a repeatable scenario-based test harness; and an evaluation summary covering detection behaviour and embedded feasibility (latency and memory footprint).

1.3.1 Scope

In scope: anomaly detection for CO_2/temperature/humidity; an end-to-end pipeline (data preparation → model development → embedded deployment → evaluation); implementation on ESP32 using MicroPython; simulation and hardware testing with resource-aware reporting.
Out of scope: full greenhouse control automation (closed-loop actuation), cloud dashboards, multi-site deployment infrastructure, comprehensive agricultural decision support, and automated fault repair.

1.3.2 Audience

The primary audience is greenhouse operators/maintainers, IoT/embedded developers, and academic assessors/supervisor who require a clearly motivated project with explicit objectives, a coherent product vision, and evidence-based evaluation aligned with the module assessment criteria [20].

2 Background review

2.1 Greenhouse monitoring context and why “abnormal” is hard to define

A greenhouse environment is designed but not necessarily a homogenous one. Temperature, humidity, and CO_2 gradients may coexist inside ventilated greenhouse structures [1]. Data readings across points can very well vary, and control moves can cause abrupt transient responses. This implies that the assumption underlying a simple detector, “outlier = anomaly,” may result in too many false alarms for the anomaly-detection problem. A detector relevant to a greenhouse atmosphere must thus include the evaluation of plausibility limits (hard limits), short-term dynamics (rate of change), persistence analysis (stuck-at model), and joint consistency with multiple variables such as interrelations of CO_2, temperature, or humidity instead.

Smart greenhouse systems increasingly adopt the integration of monitoring and control using IoT technology to optimize crop growth and resource usage [5]. Multi-sensor monitoring improves observability and control effectiveness, while also increasing system complexity and potential data defects (sensors, calibration, loss of communications, processing errors) [6]. Anomaly detection is thus most useful as a sub-module within the context of system reliability, rather than posing a replacement role within control and decision-making.

2.2 Existing greenhouse intelligence focuses on control; reliability is a complementary need

A paper by Ihoume et al. illustrates the potential for sensor data to support the construction of models such as those found in the area of TinyML on the control of the greenhouse microclimatic environment [2]. This work suggests the joint handling of CO2, temperature, or humidity data as a system, not separate signals, and highlights the importance of detecting anomalies, which can occur through inter-variable contradictions such as a sensor malfunctioning independently while others change as expected.

In many control-oriented systems, sensor input validity is assumed. This is not a valid assumption in actual implementation, and thus an anomaly detection tool is needed that annotates, gates, or prioritizes the investigation of sensor readings before they are utilized in control or reporting.

2.3 Edge/TinyML motivation: architecture trade-offs, not just “where it runs”

Edge computing for smart agriculture promotes moving computation infrastructure closer to the sensing layer to improve reaction speed and autonomy [9]. The related research area of edge-AI also argues that an IoT network benefits from local inference to decrease dependence on upstream communication [13]. Within a greenhouse setting, these factors apply to actual resilience, where local detection can keep operation going during disconnection and provide quicker maintenance response times to failed and drifting sensors.

However, the “edge” represents a continuum of devices, ranging from microcontrollers to Linux-class devices. The current project is focused on the ESP32 with MicroPython, and its feasibility needs to be proved under the resource budgets available for microcontrollers. The literature on TinyML discusses the predictability of memory consumption and low-latency inference on resource-constrained devices [3]. Research related to ML on typical microcontrollers supports the notion that embedded ML is practical; nonetheless, the robustness and support for the practice are model-family- and toolchain-dependent [7]. Thus, the selection of the approach needs to emphasize resource budgets, combined with the quality of detection.

2.4 Anomaly detection approaches suitable for constrained devices

Occurrences that are known as anomalies are not always plentiful. The reason for this is that the process of anomaly identification with faults in a greenhouse setup requires carefully organized experiments or maintenance data that may not be exhaustive.

Antonini et al. describe an adjustable, unsupervised TinyML anomaly detection solution suitable for embedded systems' strict parameters [4]. By implication, embedded systems for anomaly detection can perform effectively, given the appropriate choices for constricted execution, including the flexibility to handle dynamically changing conditions. Autoencoder methods can perform anomaly detection based on the reconstruction loss. The feasibility of using deep learning for real-time anomaly detection is provided via the autoencoder framework by Sai Charan for TinyML, although these are typically more computationally intensive and pose more integration risks with constricted systems [14]. Other autoencoder TinyML implementations for IoT anomaly detection also highlight increased complexities compared to basic solutions [12]. The consideration for autoencoder solutions will continue for the project, requiring justifications for feasibility on the ESP32 platform.

2.4.1 Approach trade-offs for this project

Given constrained execution on ESP32 and limited labelled anomalies, this project compares three practical families: (i) a threshold/rule baseline, (ii) a lightweight unsupervised model

(e.g., one-class or isolation-style scoring on engineered features), and (iii) a tiny autoencoder. Feasibility depends on predictable compute/memory costs and deployment simplicity on microcontrollers [3], [7], while TinyML anomaly detection literature supports unsupervised methods under constrained settings [4]. Autoencoder-based approaches can be effective but must be simplified and profiled early to be realistic on MicroPython + ESP32 [12], [14].

Approach	Labels needed	Compute	Memory	Interpretability	Robustness to drift
Threshold / rule baseline (bounds, rate-of-change, persistence)	None	Low	Low	High	Medium (retuning needed)
Lightweight unsupervised model (one-class / isolation-style scoring on features)	None (or minimal)	Low–Medium	Low–Medium	Medium (feature-driven)	Medium–High (can update statistics/features)
Tiny autoencoder (reconstruction error)	Mostly "normal" data	Medium–High	Medium–High	Low–Medium	Medium (may need retraining/updates)

Implication: implementation will start with a robust baseline detector to establish an end-to-end pipeline and testing evidence, then evaluate whether a lightweight unsupervised model provides measurable benefit within ESP32 constraints. A tiny autoencoder is treated as higher risk and will only be pursued if profiling shows feasibility and a clear performance gain [3], [4], [7], [12], [14].

2.5 Deployment and optimisation: why profiling matters

In the field of embedded machine learning projects, deployment challenges are common, whereby models that are acceptable in development environments do not run satisfactorily on devices because of runtime issues or unsupported calculations. In TinyML circles, there is an emphasis on inference techniques and tool sets that are suitable for deployment in an embedded system [3], [7]. Evaluation studies comparing quantized inference methods have found that it is difficult to generalize results because architecture dependence must fall back on empirical analysis [8]. In the ongoing research, it has become necessary that detection behavior, including scenario detection and false alarms, and embedding viability in terms of latency and memory requirements on an ESP32 microcontroller need to be accounted for by

presenting relevant profiles [3], [7], [8].

2.6 Engineering workflow: simulation-first development and repeatability

A wokwi simulation makes microcontroller experiments easily repeatable, yet reduces the need for immediate access to hardware [10]. It allows a simulation environment to include deterministic anomaly simulation (spike, dropout, drift, stuck-at, out-of-range) with end-to-end pipeline validation (ingestion to inference to signalling) necessary for a system integration challenge, such as embedded anomaly detection, which requires validation of microcontroller execution time, memory, and compatibility between MicroPython, code, and I/O interfaces at all levels and points.

2.7 Research gap and contribution

The Greenhouse literature is focused on observation and control for optimization purposes [5,6], and multivariate modeling for embedding intelligence [2]. In the TinyML literature, there is focus on inferability under microcontroller limitations [3,7], and on anomaly detection architectures amenable to embedding [4,14]. There appears to be less emphasis on a greenhouse-specific anomaly detection module optimized and developed as a self-contained microcontroller-friendly construct, with rigorous testing and profiling as part of the evaluation process. The specific physical environment and requirements of a greenhouse are likely to require this sort of focused attention. This project contributes:

- A greenhouse-relevant anomaly detection pipeline that accounts for microclimate variation and multivariate behaviour [1], [2].
- A constrained-device implementation plan that treats memory/latency as first-class evaluation criteria [3], [7], [8].
- A repeatable development and evaluation workflow using simulation-first testing followed by ESP32 validation [10].

3 Technical progress and plan (Semester 2, Week 1–Week 12)

3.1 Progress summary (current status)

As of the date of this writing, it has accomplished the necessary scoping and engineering decisions necessary for starting its implementation. The final methodology is as follows: ESP32 is the target hardware, the runtime environment is under supervision by MicroPython, the approach will use the simulation-first method with Wokwi and PlatformIO, and the proposed deployment strategy is suited for embedded inference with limited resources [3], [7], [10] and is more appropriate for the knowledge-based goal of emlearn [8], [20].

3.2 Updated plan and milestones (for the Gantt chart)

Milestones:

- M1 (W2): Wokwi ESP32 baseline loop running (simulated CO_2/temp/RH + serial logging) [10].
- M2 (W4): end-to-end pipeline skeleton in simulation (windowing/features + anomaly output).
- M3 (W6): candidate lightweight model trained and exportable via emlearn-oriented workflow [7].
- M4 (W8): integrated inference validated on ESP32 (simulation → hardware).
- M5 (W10): evaluation complete (detection results + latency/memory profiling) [8].
- M6 (W12): report finalised with evidence and appendices [20].

Work packages (Gantt rows):
WP1 requirements & success criteria (W1–W2)
WP2 data pipeline + preprocessing plan (W1–W3)
WP3 Wokwi baseline MicroPython loop (W1–W2) [10]
WP4 test harness + anomaly injection scenarios (W2–W4)
WP5 baseline detector (W3–W5)
WP6 model training & selection (W4–W6) [3], [7]
WP7 model export + MicroPython integration (W5–W8) [7]
WP8 hardware validation on ESP32 (W7–W9)
WP9 profiling + evaluation (W8–W10) [8]
WP10 report writing (W1–W12) [20]

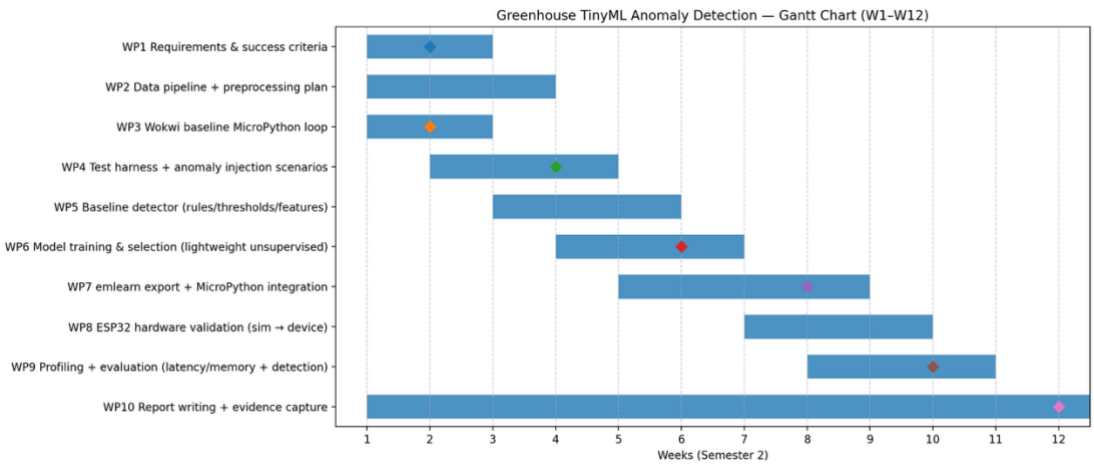

Figure 1. Updated Gantt chart for Semester 2 (Weeks 1–12). Horizontal bars show planned

work-package durations (WP1–WP10). Diamond markers indicate milestone deadlines (M1–M6) aligned to the plan in Section 3.2.

3.3 System requirements and success criteria (traceable and testable)

Functional requirements:
FR1 ingest (or simulate) CO_2, temperature, and relative humidity samples on a fixed sampling schedule.
FR2 maintain rolling windows and compute lightweight features needed for anomaly detection (bounds, rate-of-change, rolling statistics).
FR3 output an anomaly decision (binary flag and/or anomaly score) for each update window.
FR4 provide clear anomaly outputs suitable for maintenance/monitoring (serial logs + optional alert).
FR5 support repeatable/controlled anomaly inputs in simulation to enable regression testing [10].

Non-functional requirements:
NFR1 latency: inference + decision should support near-real-time detection at the chosen sampling rate; latency will be measured and reported on ESP32 [8].
NFR2 memory footprint: the solution must remain feasible within ESP32 constraints; memory use will be profiled and reported [3], [7].
NFR3 robustness: avoid excessive false alarms during physically plausible microclimate change (including gradients) [1], [6].
NFR4 maintainability: modular pipeline (ingest, features, model, output) to support iteration and debugging [20].

3.4 Test plan (simulation-first → hardware validation)

Test environments:

- Simulation: Wokwi ESP32 project for controlled scenario testing [10].
- Hardware: ESP32 running MicroPython for feasibility and profiling.

Scenario-based tests:

- Normal operation baseline
- Spike anomaly
- Dropout anomaly
- Stuck-at anomaly

- Drift anomaly
- Out-of-range anomaly
- Cross-variable inconsistency (motivated by multivariate greenhouse modelling) [2]

Metrics:

- Detection behaviour by anomaly type (detections and false alarms)
- Embedded feasibility: loop/inference latency on ESP32, memory footprint proxies, and continuous-runtime stability observations [7], [8]

3.5 Implementation/prototype plan

Planned pipeline: input (sensor/simulation) → rolling window buffers → feature extraction → inference (baseline detector first, then model) → anomaly decision/signalling → logging for evaluation. An emlearn-based representation will be used to support lightweight inference compatible with microcontroller constraints and to reduce deployment mismatch risk [7].

Staged build strategy:

- Stage A (W1–W2): baseline loop + logging in Wokwi (logs/screenshots) [10].
- Stage B (W2–W4): anomaly injection + pipeline skeleton (pass/fail per scenario).
- Stage C (W4–W8): model training + export + MicroPython integration (consistent outputs across desktop validation, simulation, and hardware) [3], [7].
- Stage D (W8–W10): profiling + evaluation write-up (measured latency/memory + detection outcomes) [8].

3.6 Issues, constraints, and immediate next steps

One major constraining factor related to the delivery aspect has been the level of integration and profiling required by the limited MicroPython+ESP32 platform. This has been addressed through a simulation-first approach involving a staged build process, providing preliminary results related to the baseline loop, the anomaly injection harness, and the pipeline scaffolding before the model choice and integration [10], [20]. Within MicroPython, internal constraints related to the model size can also pose a limitation; addressing it will require emphasis on the conservative feature evaluation, memory, and profiling [3], [7].

Immediate next steps:

1. Define sampling/window assumptions and measurable success criteria (W1–W2).
2. Implement the Wokwi ESP32 MicroPython loop with serial logging (W1–W2) [10].

3. Implement anomaly injection scenarios + expected outcomes (W2–W4).
4. Deliver a baseline lightweight detector (W3–W5), then begin model export/integration (W4–W8) [3], [7].

4 Professionalism and risk

4.1 Risk management approach

Risk management will be carried out as a continuous process. The risks will be analyzed on a weekly basis in accordance with the plan in Semester 2, and the evidence in GitHub commit and issues logs, test logs, profiling notes, and the revised Gantt chart will be used for this purpose [20]. This provides traceability in terms of decision-making processes and assures that the report does not depend on assumptions but on realistic constraints.

4.2 Key risks and mitigation

- R1 Schedule compression (Med/High): staged simulation-first deliverables and weekly GitHub issues aligned to milestones [10], [20]. Contingency: prioritise a complete end-to-end baseline detector with evaluation; treat advanced models as stretch goals [3], [7].
- R2 ESP32 + MicroPython performance limits (Med/High): lightweight features, bounded windows, and early profiling; contingency: simplify detection logic and document constraints clearly [3], [7], [20].
- R3 Deployment mismatch (Med/High): select embedded-feasible model families and validate export/integration early; contingency: adopt a simpler deployable approach and justify trade-offs [3], [7], [10].
- R4 Scenario realism (Med/Med): cover common fault modes and add multivariate checks; contingency: report results as scenario-based reliability testing and add hardware run logs as realism evidence [2], [6], [20].
- R5 Simulation vs hardware differences (Med/Med): validate on ESP32 hardware once the pipeline is stable; contingency: replay logged data on-device and document limitations [10], [20].

4.3 Legal, ethical, environmental, IP and EDI considerations

Flowcharted text

Data protection: The proposed project uses greenhouse variables such as carbon dioxide concentration, temperature, and humidity, none of which are personally identifiable. The best

practices for origin, storage, and record-keeping will be followed (cf. [20]).

Safety: ESP32-based work, despite being at lower voltage, demands correct wiring and supply handling practices. The simulation-based development method guards against risks associating with early hardware development (references [10], [20]).

Social Impact: Early warning can result in less maintenance and more reliability, but false warnings can result in alert fatigue. The analysis will monitor the false alarm rate and explain the reasons for the decisions on the thresholds (references [6], [20]).

Environmental issues:

On-device inference may reduce reliance on constant cloud connectivity and help achieve low-power consumption goals (refs. [3], [7]).

Professional Practice (GitHub): The code base, simulation projects, test cases, and evidence of activity (logs, screenshots, and profiler comments) are to be kept within a GitHub repository, thereby creating a record of progress against expectations of the module (see [20]).

Intellectual property/licensing:

The original intellectual property includes the MicroPython application source code, preprocessing scripts, model artifacts (if produced), evaluation results, and text report. Third-party libraries/platforms (including MicroPython, Wokwi, PlatformIO, as well as external libraries or datasets used) will be cited within the source code repository references. These references include references [10], [20].

Reference:

[1] Teitel, M., Atias, M. and Barak, M. (2010) *Gradients of temperature, humidity and CO_2 along a fan-ventilated greenhouse*. Biosystems Engineering. doi: 10.1016/j.biosystemseng.2010.03.007.

[2] Ihoume, I., Tadili, R., Arbaoui, N., Benchrifa, M., Idrissi, A. and Daoudi, M. (2022) *Developing a multi-label tinyML machine learning model for an active and optimized greenhouse microclimate control from multivariate sensed data*. Artificial Intelligence in Agriculture. doi: 10.1016/j.aiia.2022.08.003.

[3] Immonen, R. and Hämäläinen, T. (2022) *Tiny Machine Learning for Resource-Constrained Microcontrollers*. Journal of Sensors. doi: 10.1155/2022/7437023.

[4] Antonini, M., Pincheira, M., Vecchio, M. and Antonelli, F. (2023) *An Adaptable and Unsupervised TinyML Anomaly Detection System for Extreme Industrial Environments*. Sensors, 23, 2344. doi: 10.3390/s23042344.

[5] Bouarroudj, K., Babaa, F. and Touil, A. (2025) *IoT-based monitoring and control for optimized plant growth in smart greenhouses using soil and hydroponic systems*. Internet of Things, 33. doi: 10.1016/j.iot.2025.101710.

[6] Bicamumakuba, E., Reza, M.N., Jin, H., Samsuzzaman, Lee, K.-H. and Chung, S.-O. (2025) *Multi-Sensor Monitoring, Intelligent Control, and Data Processing for Smart Greenhouse Environment Management*. Sensors, 25, 6134. doi: 10.3390/s25196134.

[7] Sakr, F., Bellotti, F., Berta, R. and De Gloria, A. (2020) *Machine Learning on Mainstream Microcontrollers*. Sensors, 20, 2638. doi: 10.3390/s20092638.

[8] Julião, A., Rombo, G. and Oliveira, J.P. (2025) *Comparative Study of Quantized CNN Inference on ARM and RISC-V Microcontrollers*. In: 9th International Young Engineers Forum on Electrical and Computer Engineering (YEF-ECE). doi: 10.1109/YEF-ECE66503.2025.11117526.

[9] Hernández Hidalgo, C., González-Vidal, A. and Skarmeta, A.F. (2023) *Edge computing in Smart Agriculture scenario based on TinyML for irrigation control*. In: IEEE World Forum on Internet of Things (WF-IoT). doi: 10.1109/WF-IOT58464.2023.10539452.

[10] Asparuhova, K., Shehova, D., Asenov, S., Kanevski, H. and Parushev, A. (2024) *Using WOKWI Simulator to Support Engineering Student Learning in Microcontrollers and Sensors*. In: International Scientific Conference Electronics (ET2024). doi: 10.1109/ET63133.2024.10721553.

[11] Almufareh, M.F., Ahmad, Z., Pervez, A. and Iftikhar, M. (2024) *Edge Computing-Enhanced Machine Learning Framework with Anomaly Detection-Aware Predictive Modelling in IoT Healthcare Systems*. In: 26th International Multi-Topic Conference (INMIC). doi: 10.1109/INMIC64792.2024.11004306.

[12] Kandil, H., Bouimejane, W., Mouhcine, M. and Benaboud, H. (2025) *An Anomaly-based Intrusion Detection System for IoT Environments using Autoencoder Neural Networks and TinyML*. International Journal of Computer Applications, 187(59).

[13] Karras, A., Giannaros, A., Karras, C., Giotopoulos, K.C., Tsolis, D. and Sioutas, S. (2023) *Edge Artificial Intelligence in Large-Scale IoT Systems, Applications, and Big Data Infrastructures*. In: 8th South-East Europe Design Automation, Computer Engineering, Computer Networks and Social Media Conference (SEEDA-CECNSM). doi: 10.1109/SEEDA-CECNSM61561.2023.10470756.

[14] Sai Charan, K. (2023) *An Auto-Encoder Based TinyML Approach for Real-Time Anomaly Detection*. SAE International Journal of Advances & Current Practices in Mobility, 5(4), pp. 1496–1501. doi: 10.4271/2022-28-0406.

[15] Srisuchinwong, D., Sukhachewanon, B. and Chanwimalueang, T. (2021) *Acquiring unobtrusive sleep-related signals through an ESP32-based data logger*. In: 13th International Conference on Knowledge and Smart Technology (KST). doi: 10.1109/KST51265.2021.9415820.

[16] Bashir, M.J., Henna, S. and Furey, E. (2025) *Explainable AI for Smart Greenhouse Control: Interpretability of Temporal Fusion Transformer in the Internet of Robotic Things*. arXiv:2512.11852v1.

[17] Jeaunita, T.C.J., Ramesh, T., Kalpana, A.V. and Shanthala, P.T. (2025) *Multi-Task Deep Learning Framework for Anomaly Detection and Carbon Emission Prediction in Precision Agriculture*. IEEE Transactions on Sustainable Computing, 10(6). doi: 10.1109/TSUSC.2025.3619577.

[18] Palmieri, M., Donno, L., Cimolin, V. and Galli, M. (2023) *Cervical Range of Motion Assessment through Inertial Technology: A Validity and Reliability Study*. Sensors, 23, 6013. doi: 10.3390/s23136013.

[19] Song, Z. (2025) *Greenhouse Anomaly Detection in Environmental Sensor Data Using TinyML*. Project proposal (unpublished), October 2025.

[20] Oxford Brookes University (2025) *Computing Project Module Handbook 2025–26*. Unpublished module handbook.